Laborator 3
CURSORI

I. Obiective

4.1. Cursorul implicit

4.2. Cursorul explicit

II. Consideraţii teoretice

Atunci când se execută o comandă SQL, Oracle Server deschide o zonă de memorie (context area) în care comanda este executată. Cursorul este un pointer către această zonă

În PL/SQL se utilizează două tipuri de cursoare:

1. **implicit**: declarat pentru toate instrucţiunile PL/SQL de tip LMD (INSERT/UPDATE/DELETE);
2. **explicit**: declarat şi gestionat de programator.

4.1. CURSORUL IMPLICIT

Este declarat de PL/SQL implicit pentru toate comenzile de manipulare a datelor (INSERT, UPDATE, DELETE);

Dacă o instrucţiune LMD nu afectează nici o linie a tabelei nu se generează eroare, însă excepţia trebuie tratată folosind atributele speciale ale cursorilor;

Atributele cursorului implicit: SQL%ROWCOUNT, SQL%FOUND, SQL%NOTFOUND

Exemplu:

Se şterge un produs din tabela PRODUSE şi se contorizează numărul de rânduri şterse.

```
SET SERVEROUTPUT ON
DECLARE
        v_rez NUMBER(2);

BEGIN
        DELETE FROM produse
        WHERE id_produs=111;

        v_rez:=SQL%ROWCOUNT;
        DBMS_OUTPUT.PUT_LINE (v_rez || ' randuri sterse');
        COMMIT;
END;
/
```

Se încearcă modificarea denumirii produsului cu codul 3, în cazul în care acest produs nu există (comanda update nu realizează nici o modificare) va fi afişat un mesaj corespunzător.

```
BEGIN
      UPDATE produse
      SET denumire_produs='cafea'
      WHERE id_produs=3;
      IF SQL%NOTFOUND THEN
                  DBMS_OUTPUT.PUT_LINE('Nu exista produsul cu acest cod');
      END IF;
END;
/
```

Se şterge din tabela EMP_SAL salariatul al cărui ID este introdus de utilizator prin intermediul variabilei g_angid:

```
ACCEPT g_angid PROMPT 'Introduceti id-ul salariatului:'
VARIABLE nr_sters varchar2(100)
DECLARE
BEGIN
      DELETE FROM emp_sal WHERE id_angajat=&g_angid;
      :nr_sters:=TO_CHAR(SQL%ROWCOUNT)||' INREGISTRARI STERSE';
END;
/
PRINT nr_sters
Rollback;
```

4.2. CURSORUL EXPLICIT

- ✓ se foloseşte pentru a procesa individual fiecare linie (înregistrare) returnată de o instrucţiune SELECT ce returnează mai multe înregistrări.
- ✓ mulţimea înregistrărilor returnate de o instrucţiune SELECT este numită mulţime rezultat.
- ✓ cursorul păstrează un pointer către linia curentă în cadrul unei mulţimi rezultat.

Verificarea stării unui cursor explicit se realizează prin intermediul următoarelor atribute:

1. nume_cursor%ISOPEN - evaluat la TRUE în cazul în care cursorul este deschis;
2. nume_cursor %NOTFOUND - evaluat la TRUE în cazul în care cel mai recent FETCH nu a returnat nici o linie;
3. nume_cursor %FOUND - complementul lui %NOTFOUND;

4. nume_cursor %ROWCOUNT - are ca valoare numărul liniilor returnate până în momentul curent.

Prelucrarea cursorului explicit presupune parcurgerea următoarelor etape:

1) se declară variabilele în care vor fi încărcate valorile corespunzătoare unei linii din cursor;
2) se declară cursorul explicit, specificându-se un nume pentru acesta şi definindu-se interogarea de procesat în cadrul lui:

DECLARE nume_cursor IS SELECT.....;

3) se deschide cursorul prin intermediul instrucţiunii OPEN, care execută interogarea şi legarea tuturor variabilelor referite. Înregistrările returnate de interogare sunt desemnate drept set activ de date, care pot fi de acum încărcate.

OPEN nume_cursor;

4) utilizându-se instrucţiunea FETCH, se încarcă linia curentă din cursor în variabile. Fiecare încărcare determină mutarea pointerului cursorului la linia următoare din setul activ de date.

FETCH nume_cursor INTO var1, var2,.....;

5) este închis cursorul prin instructiunea CLOSE, care dezafectează setul activ de linii.

Cursorul poate fi din nou deschis pentru a stabili un nou set activ de linii.

CLOSE nume_cursor;

Pentru a procesa liniile unui cursor explicit se defineşte de obicei o buclă pentru executarea unui FETCH în fiecare iteraţie. În final, toate liniile din setul activ sunt procesate şi un FETCH executat fără succes poziţionează atributul %NOTFOUND pe TRUE.

Înaintea primului FETCH, %NOTFOUND se evaluează la NULL, ca şi în cazul în care FETCH nu se execută niciodată cu succes.

Exemple:

Să se afişeze lista cu numele şi salariul angajaţilor din departamentul 60 folosind un cursor explicit.

Pentru o flexibilitate mai mare se poate utiliza o variabilă de tip record pentru încărcarea valorilor din cursor. Această variabilă de tip record poate avea aceleaşi atribute ca şi cursorul prin specificarea proprietăţii %ROWTYPE. În acest caz încărcarea din cursor se va face direct prin instrucţiunea fech var_cursor into var_record.

Secvența de cod este:

```
set serveroutput on
DECLARE
     cursor ang_cursor is select id_angajat, nume, salariul
     from angajati where id_departament=60;
     ang_id angajati.id_angajat%type;
     ang_nume angajati.nume%type;
     ang_sal angajati.salariul%type;
BEGIN
     dbms_output.put_line('Lista cu salariariile angajatilor din departamentul 60');
     open ang_cursor;
loop
     fetch ang_cursor into ang_id, ang_nume, ang_sal;
     exit when ang_cursor%notfound;
     dbms_output.put_line('Salariatul '||ang_nume||' are salariul: '||ang_sal);
   end loop;
   close ang_cursor;
end;
/
```

Exemplul de mai sus poate fi rescris astfel:

```
set serveroutput on
DECLARE
       cursor ang_cursor is select id_angajat, nume, salariul
       from angajati where id_departament=60;
       --tipul record pentru incarcarea valorilor
       cursorului ang_rec ang_cursor%rowtype;
 BEGIN
     dbms_output.put_line('Lista cu salariariile angajatilor din departamentul 60');
     open ang_cursor;
     loop
         fetch ang_cursor into ang_rec;
         exit
         when ang_cursor%notfound;
         dbms_output.put_line('Salariatul'||ang_rec.nume||' are  salariul:
         '||ang_rec.salariul);
     end loop;
     close ang_cursor;
end;
/
```

Să se încarce în tabela MESAJE primii 5 angajaţi (id şi nume)

```
CREATE TABLE mesaje ( cod varchar2(7), nume varchar2(20) );

DECLARE
        v_id angajati.id_angajat%type;
        v_nume angajati.nume%type;
        CURSOR c1 IS SELECT id_angajat, nume FROM angajati;
BEGIN
        OPEN c1;
        FOR i IN 1..5 LOOP
                FETCH c1 INTO v_id, v_nume;
                INSERT INTO mesaje VALUES(v_id, v_nume);
        END LOOP;
        CLOSE c1;
END;
/
SELECT * FROM mesaje;
```

Testul de ieşire din buclă în acest caz se poate face şi cu ajutorul atributului %ROWCOUNT:

Delete from mesaje;

```
DECLARE
        v_id angajati.id_angajat%type;
        v_nume angajati.nume%type;
        CURSOR c1 IS SELECT id_angajat, nume FROM angajati;

BEGIN
        OPEN c1;
        LOOP
                FETCH c1 INTO v_id, v_nume;
                EXIT WHEN c1%ROWCOUNT>5 OR c1%NOTFOUND;
                INSERT INTO mesaje VALUES (v_id, v_nume);
        END LOOP;
        CLOSE c1;
END;
/
    SELECT * FROM mesaje;
```

Să se afişeze primele 3 comenzi care au cele mai multe produse comandate. În acest caz înregistrările vor fi ordonate descrescător în funcţie de numărul produselor comandate:

```
SET SERVEROUTPUT ON
DECLARE
   CURSOR c_com IS
   select c.nr_comanda, count(r.id_produs)
     Numar from comenzi c, rand_comenzi r
     where c.nr_comanda=r.nr_comanda
   group by c.nr_comanda
   order by count(r.id_produs) desc;

   rec_com c_com%rowtype;

BEGIN
   DBMS_OUTPUT.PUT_LINE('Numarul de produse pentru fiecare comanda:');
   IF NOT c_com%ISOPEN THEN
         OPEN c_com;
   END IF;
   LOOP
         FETCH c_com INTO rec_com;
         EXIT
         WHEN c_com%NOTFOUND OR c_com%ROWCOUNT>3;
         DBMS_OUTPUT.PUT_LINE('Comanda'||rec_com.nr_comanda||' are:
               '||rec_com.numar||' produse');
   END LOOP;
   CLOSE c_com;
END;
 /
```

4.2.1

Gestiunea implicită a cursorului prin utilizarea unui ciclu FOR:

```
FOR nume_record IN nume_cursor LOOP
      ....
END LOOP;
```

În acest caz, tipul RECORD nu trebuie declarat. Se realizează în mod implicit deschiderea, încărcarea şi închiderea cursorului.

Exemplu

Se afişează printr-un ciclu FOR numele şi salariile angajaţilor din departamentul 60:

```
set serveroutput on
declare
        cursor ang_cursor is
        select id_angajat, nume, salariul
        from angajati
        where id_departament=60;
begin
        dbms_output.put_line('Lista cu salariariile angajatilor din
departamentul 60');
        for ang_rec in ang_cursor loop
                dbms_output.put_line('Salariatul      '||ang_rec.nume||'      are
        salariul: '||ang_rec.salariul);
        end loop;
end;
/
```

Utilizarea unui cursor direct în cadrul instrucţiunii FOR.

În acest caz cursorul nu este declarat, nu are nume, este reprezentat doar de interogarea SELECT din cadrul instrucţiunii FOR, astfel:

```
FOR NUME_RECORD IN (SELECT.....) LOOP
        ....
END LOOP;
```

Dezavantajul în acest caz este că nu se pot utiliza atributele cursorului din cauza faptului că acesta nu are nume.

Exemplu

Să se afişeze suma aferentă salariilor din fiecare departament:

```
set serveroutput on
declare
        begin
        dbms_output.put_line('Total salarii pe fiecare
        departament:'); for dep_rec in
                (select d. id_departament dep, sum(a.salariul)
                sal from angajati a, departamente d
                where a.id_departament=d.id_departament
                group by d.id_departament)
        loop
                dbms_output.put_line('Departamentul '||dep_rec.dep||' are de
        platit salarii in valoare de: '||dep_rec.sal||' RON');
                end loop;
end;
/
```

Utilizarea cursorului cu parametru

Pentru o flexibilitate mai mare se pot declara şi utiliza cursori cu parametru care transmit valorile parametrilor actuali în cererile SQL. Declararea cursorului cu parametru se face astfel:

Cursor nume_cursor (parametru1 tip_data,.....)
Is select ....;

Deschidere:

Open nume_cursor(valoare_parametru1,.....);

Cursoarele parametrizate nu oferă o funcţionalitate suplimentară ci doar o modalitate simplă şi clară de a specifica valori de intrare.

Tipurile parametrilor sunt scalare, dar nu li se precizează dimensiunea; ele fiind referite în interogare.

Exemple:

Să se afişeze produsele al căror cantitate totală comandată este mai mare decât o valoare primită drept parametru.

```
SET SERVEROUTPUT ON

DECLARE
    CURSOR c_prod (p_val NUMBER) IS
    SELECT p.id_produs, p.denumire_produs, sum(r.cantitate) total
    FROM produse p, rand_comenzi r
    WHERE p.id_produs =r.id_produs
    GROUP BY p.id_produs, p.denumire_produs
    HAVING sum(r.cantitate)>p_val
    ORDER BY total desc;
    v_val NUMBER(5);
    rec_prod c_prod%rowtype;
BEGIN
    v_val:=500;
    DBMS_OUTPUT.PUT_LINE('Produsele al caror cantitate vândută este mai mare
    decat '|| v_val);
    IF NOT c_prod%ISOPEN THEN
        OPEN c_prod (v_val);
  END IF;
        LOOP
    FETCH c_prod into rec_prod;
    EXIT WHEN c_prod%notfound;
    DBMS_OUTPUT.PUT_LINE('Din produsul '||rec_prod.id_produs||',
    '||rec_prod.denumire_produs||', s-au vandut ' ||rec_prod.total||' unitati');
  END LOOP;
  CLOSE c_prod;
END;
/
```

Să se afişeze pentru fiecare comanda produsele comandate. În acest caz se utilizează două variabile de tip cursor:

```
SET SERVEROUTPUT ON
DECLARE
--cursorul care va prelua comenzile incheiate
    CURSOR c_com IS
    SELECT nr_comanda, data
    FROM comenzi
    Where modalitate= 'online'
    ORDER BY nr_comanda;

--cursorul care, pentru fiecare comanda, va afisa produsele din cadrul
acesteia, ordonate descrescator
    CURSOR c_prod (p_nr_comanda NUMBER) IS
    SELECT r.id_produs, p.denumire_produs,
    r.cantitate FROM produse p, rand_comenzi r
    WHERE p.id_produs=r.id_produs
    AND r.nr_comanda=p_nr_comanda
    ORDER BY r.id_produs desc;

    rec_com c_com%rowtype; --variabila record pentru campurile din
    primul cursor
    rec_prod c_prod%rowtype; --variabila record pentru campurile din
    al doilea cursor

BEGIN
    OPEN c_com;
    LOOP
      FETCH c_com into rec_com;
      EXIT WHEN c_com%notfound;
      DBMS_OUTPUT.PUT_LINE('Comanda '|| rec_com.nr_comanda ||'
      incheiata la data de '||rec_com.data);

      OPEN c_prod (rec_com.nr_comanda); --cursorul primeste drept
      parametru numarul comenzii care a fost afisata
      LOOP
           FETCH c_prod into rec_prod;
           EXIT WHEN c_prod%notfound;
           DBMS_OUTPUT.PUT_LINE('Din produsul '|| rec_prod.id_produs||',
           '||rec_prod.denumire_produs||', s-au comandat ' ||rec_prod.cantitate||'
           bucati');
      END LOOP;
    CLOSE c_prod;

    DBMS_OUTPUT.PUT_LINE('============');
    END LOOP;
    CLOSE c_com;
END;
/
```

Actualizarea înregistrărilor returnate de cererea cursorului. Clauza FOR UPDATE

- se blochează setul de înregistrări ale cursorului în 2 variante: NOWAIT şi WAIT n:

```
CURSOR C IS SELECT .... FROM....
FOR UPDATE [OF COLUMN_NAME] [NOWAIT|WAIT n];
```

- se adaugă clauza FOR UPDATE în interogarea asociată cursorului pentru a bloca liniile afectate atunci când cursorul este deschis.
- clauza NOWAIT - determină apariţia unei erori dacă liniile sunt blocate de o altă sesiune.
- când mai multe tabele sunt implicate în interogare, se poate folosi FOR UPDATE pentru a impune blocarea liniilor unei tabele anume. Liniile unei tabele sunt blocate numai în cazul în care clauza FOR UPDATE face o referire la o coloană din acea tabelă.

Exemplu:

Se creează tabela Situatie care pastreaza informatii despre comenzi: codul, valoarea comenzii. Se adaugă în aceasta coloana TVA, care va păstra valoarea TVA pentru fiecare comandă. Se creează un cursor căruia i se adaugă clauza FOR UPDATE pentru a bloca liniile afectate din tabelă, atunci când cursorul este deschis, iar pentru fiecare comandă din cursor se va calcula valoarea TVA

```
DROP TABLE situatie;
CREATE TABLE situatie AS
SELECT c.nr_comanda cod, SUM(r.cantitate*r.pret) as valoare
FROM comenzi c, rand_comenzi r
WHERE c. nr_comanda =r. nr_comanda
AND c.modalitate= 'online'
GROUP BY c. nr_comanda;
ALTER TABLE situatie
ADD(tva NUMBER(10));
DECLARE
CURSOR c_situatie IS
SELECT cod, valoare, tva
FROM situatie
FOR UPDATE OF tva NOWAIT;
BEGIN
FOR rec_situatie IN c_situatie
LOOP

   UPDATE situatie
   SET tva=valoare*0.19
   WHERE cod=rec_situatie.cod;
   DBMS_OUTPUT.PUT_LINE('Comanda'||rec_situatie.cod||'   are
   valoarea     totala de'||rec_situatie.valoare||'RON si TVA de:
   '||rec_situatie.tva );
END LOOP;
END;
/
```

Atenţie: În cazul de mai sus se poate observa faptul că atributul *REC_SITUATIE.TVA* este NULL după execuţia comenzii *UPDATE* deoarece nu se actualizează automat şi cursorul odată cu tabela. Acesta trebuie închis şi redeschis pentru a fi vizibile actualizările din tabelă.

Pentru manipularea cât mai uşoară a comenzilor LMD UPDATE şi DELETE se poate utiliza clauza WHERE CURRENT OF care permite actualizarea înregistrărilor pe baza liniei curente din cursor.

```
UPDATE tabela
SET camp=....
WHERE CURRENT OF nume_cursor;
```

- se poate referi linia din tabela originară, pentru actualizare sau ştergere, prin intermediul liniei curente a cursorului (cea procesată de ultima instrucţiune FETCH).
- clauza FOR UPDATE trebuie inclusă în definiţia cursorului pentru a bloca liniile în momentul execuţiei instrucţiunii OPEN.

Exemplu:

Exemplu de mai sus poate fi rescris, actualizarea înregistrărilor din tabela SITUATIE realizându-se cu clauza WHERE CURRENT OF:

```
DROP TABLE situatie;
CREATE TABLE situatie AS
SELECT c.nr_comanda cod, SUM(r.cantitate*r.pret) as valoare
FROM comenzi c, rand_comenzi r
WHERE c. nr_comanda =r. nr_comanda
AND c.modalitate= 'online'
GROUP BY c. nr_comanda;
ALTER TABLE situatie
ADD(tva NUMBER(10));
DECLARE
CURSOR c_situatie IS
SELECT cod, valoare, tva
FROM situatie
FOR UPDATE OF tva NOWAIT;
BEGIN
FOR rec_situatie IN c_situatie
LOOP
   UPDATE situatie
   SET tva=valoare*0.19
   WHERE CURRENT OF c_situatie;
   DBMS_OUTPUT.PUT_LINE('Comanda '||rec_situatie.cod||' are
   Valoarea totala de '||rec_situatie.valoare||' RON si tva de:
   '||rec_situatie.tva );
END LOOP;
END;
/
```

Să se afişeze lista cu numele şi salariul angajaţilor din departamentul 60 folosind un cursor explicit

```
set serveroutput on
DECLARE
        cursor ang_cursor is
        select id_angajat, nume, salariul
        from angajati where
        id_departament=60;
        ang_id angajati.id_angajat%type;
        ang_nume angajati.nume%type;
        ang_sal angajati.salariul%type;
BEGIN
        dbms_output.put_line('Lista cu salariariile angajatilor din departamentul 60');
        open ang_cursor;
        loop
                fetch ang_cursor into ang_id, ang_nume,
                ang_sal; exit when ang_cursor%notfound;
                dbms_output.put_line('Salariatul '||ang_nume||' are salariul: '||ang_sal);
        end loop;
        close ang_cursor;
end;
```

Exemplu de utilizare a instructiunii IF:

```
declare
   v_loc varchar2(10) := 'NEW YORK';
   v_nr          number(2);
begin
   select deptno
   into v_nr
   from dept
   where loc=v_loc;
   dbms_output.put_line('Numarul departamentului este:
   '||v_nr);
   if (v_nr=20) then
        dbms_output.put_line('Este 20');
        elsif v_nr < 20 then
        dbms_output.put_line('Mai mic decat 20');
   else
        dbms_output.put_line('Mai mare decat 20');
   end if;
exception
      when no_data_found then
        dbms_output.put_line('Nu exista acest departament');
      when too_many_rows then
        dbms_output.put_line('Sunt mai multe');
      when others then
        dbms_output.put_line('Eroare nespecificata');
end;
```

Exemplu de utilizare a instructiunii LOOP:

```
declare
   v_contor number(2) :=5;
   v_deptno number;
   v_dname varchar2(10);
   v_loc varchar2(10);
begin
     loop
        begin
        select deptno, dname, loc
        into v_deptno, v_dname, v_loc
        from dept
        where deptno=v_contor;
        dbms_output.put_line('Dep.cu cod '|| v_contor||' se numeste '||v_dname||' si este in
        '||v_loc);
        exception
        when no_data_found then
        dbms_output.put_line('Nu exista departamentul cu cod '||
        v_contor); end;
        v_contor := v_contor + 5;
        exit when v_contor > 70;
     end loop;
exception
     when others then
       dbms_output.put_line('Exceptie');
end;
```

III. Desfășurarea lucrării

Testarea secvențelor utilizând baza de date personală.

Probleme propuse spre rezolvare

1. Afişaţi toţi angajaţii şi comenzile încheiate de fiecare dintre aceştia. Folosiţi un cursor pentru a încărca numele angajaţilor şi un cursor parametrizat pentru încărcarea comenzilor încheiate de aceştia.

2. Afişaţi informaţii despre primele 3 comenzi care au cea mai mare valoare.

3. Afişaţi informaţii despre primii 5 salariaţi angajaţi (se va realiza filtrarea în funcţie de câmpul Data_Angajare).

Creaţi un bloc PL/SQL care determină cele mai mari 5 salarii, urmând paşii descrişi în continuare:

a) creaţi un tabel TOP_SALARII_PNU, având o coloană sal.

b) În secţiunea declarativă a blocului PL/SQL se vor declara 2 variabile: v_num de tip NUMBER (initializata cu valoarea 5) şi v_sal de tipul coloanei sal. Se va declara un cursor emp_cursor pentru regăsirea salariilor în ordine descrescătoare (se presupune că nu avem valori duplicate).

c) Se vor introduce cel mai bine plătiţi n angajaţi în tabelul TOP_SALARII_PNU;

d) Afişaţi conţinurtul tabelului TOP_SALARII_PNU.

e) Testaţi cazuri speciale, de genul n = 0 sau n mai mare decât numărul de angajaţi.

Se vor elimina înregistrările din tabelul TOP_SALARII_PNU după fiecare test.

Soluţie:

```
DECLARE
        v_num   NUMBER(3) := 5;
        v_sal     emp.sal%TYPE;
CURSOR emp_cursor IS
                SELECT DISTINCT sal
                FROM    emp
                ORDER BY sal DESC;
BEGIN
        OPEN emp_cursor; -- folositi si alte variante de lucru cu cursorul !!!!
        FETCH emp_cursor INTO v_sal;
        WHILE emp_cursor%ROWCOUNT <= v_num AND emp_cursor%FOUND
                LOOP INSERT INTO top_salarii_pnu (sal)
                VALUES (v_sal);
                FETCH emp_cursor INTO v_sal;
        END LOOP;
        CLOSE emp_cursor;
END;

Select * From TOP_SALARII_PNU
ROLLBACK
```

Cursoare cu parametru

1. Să se declare un cursor cu un parametru de tipul codului departamentului, care regăseşte numele şi salariul angajaţilor din departamentul respectiv, pentru care nu s-a specificat comisionul. Să se declare o variabilă v_nume de tipul unei linii a cursorului. Să se declare două tablouri de nume (v_tab_nume), respectiv salarii (v_tab_sal). Să se parcurgă liniile cursorului în două moduri: regăsindu-le în v_nume sau în cele două variabile de tip tablou.

Soluţie:

```
DECLARE
  CURSOR c_nume (p_idDep emp.deptno%TYPE) IS
    SELECT ename, sal
    FROM   emp
    WHERE comm IS NULL
     AND    deptno= p_idDep;
  v_nume c_nume%ROWTYPE;
  -- sau
  /* TYPE t_nume IS RECORD
       (ename emp.empno%TYPE,
        sal    emp.sal%TYPE
    v_nume t_nume;
  */
  TYPE t_tab_nume IS TABLE OF emp.ename%TYPE;
  TYPE t_tab_sal  IS TABLE OF emp.sal%TYPE;
  v_tab_nume t_tab_nume;
  v_tab_sal t_tab_sal;
BEGIN
  IF c_nume%ISOPEN THEN
    CLOSE c_nume;
  END IF;
```

```
  OPEN c_nume (20);
  FETCH c_nume INTO v_nume;
  WHILE c_nume%FOUND LOOP
    DBMS_OUTPUT.PUT_LINE (‘ Nume:’ || v_nume.ename || ‘ salariu : ‘ || v_nume.sal);
    FETCH c_nume INTO v_nume;
  END LOOP;
  CLOSE c_nume;
  -- eroare INVALID CURSOR
  --  FETCH c_nume INTO v_nume;
  DBMS_OUTPUT.PUT_LINE (‘ SI o varianta mai eficienta’);
  OPEN c_nume (30);
  FETCH c_nume BULK COLLECT INTO v_tab_nume, v_tab_sal;
  CLOSE c_nume;
  FOR i IN v_tab_nume.FIRST..v_tab_nume.LAST LOOP
    DBMS_OUTPUT.PUT_LINE (i || ‘ Nume:’ || v_tab_nume(i) || ‘ salariu : ‘ || v_tab_sal(i));
  END LOOP;
END;
/
```

2. Să se rezolve exerciţiul 1. utilizând comanda LOOP.

```
DECLARE
  CURSOR c_nume (p_idDep emp.deptno%TYPE) IS
    SELECT ename, sal
    FROM  emp
    WHERE commIS NULL
     AND deptno= p_idDep;
  v_nume c_nume%ROWTYPE;
  -- sau
  /* TYPE t_nume IS RECORD
       (ename emp.empno%TYPE,
        sal    emp.sal%TYPE
    v_nume t_nume;
  */
BEGIN
  OPEN c_nume (20);
  LOOP
    FETCH c_nume INTO v_nume;
    EXIT WHEN c_nume%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE (‘ Nume: ‘ || v_nume.ename || ‘ salariu : ‘ || v_nume.sal);
  END LOOP;
  CLOSE c_nume;
END;
/
```

3. Să se rezolve exerciţiul 1. folosind comanda FOR specifică lucrului cu cursoare.

Obs: La cursoare, comanda FOR realizează deschidere, încărcare şi închidere automată.

```
DECLARE
  CURSOR c_nume (p_idDep emp.deptno%TYPE) IS
    SELECT ename, sal*12 sal_an
    FROM   emp
    WHERE commIS NULL
    AND    deptno= p_idDep;
BEGIN
  FOR v_rec IN  c_nume (20) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE (‘ Nume:’ || v_rec.ename || ‘ salariu : ‘ || v_rec.sal_an);
  END LOOP;
END;
/
```

4. Creaţi un tabel MESAJE_PNU având o coloană rezultat de tip VARCHAR2(75). Utilizând un cursor parametrizat să se obţină codurile angajaţilor din fiecare departament şi pentru fiecare job. Rezultatele să fie inserate în tabelul MESAJE_PNU, sub forma câte unui şir de caractere obţinut prin concatenarea valorilor celor 3 coloane.

```
DECLARE
  v_cod_dep dept.deptno%TYPE;
  v_cod_job  emp.job_id%TYPE;
  v_mesaj    VARCHAR2(75);
  CURSOR dep_job IS
    SELECT DISTINCT deptno, job                – de ce e nevoie de „DISTINCT”?
    FROM    emp_pnu;
  CURSOR emp_cursor (v_id_dep NUMBER, v_id_job VARCHAR2) IS
    SELECT  empno || ‘ ‘ || deptno|| ‘ ‘ || job_id
    FROM    emp_pnu
    WHERE   deptno= v_id_dep
    AND     job= v_id_job;
BEGIN
  DELETE FROM mesaje_pnu;
  OPEN dep_job;
  LOOP
    FETCH dep_job INTO v_cod_dep,v_cod_job;
    EXIT WHEN dep_job%NOTFOUND;
    IF emp_cursor%ISOPEN THEN
      CLOSE emp_cursor;
    END IF;
    OPEN emp_cursor (v_cod_dep, v_cod_job);
    LOOP
      FETCH emp_cursor INTO v_mesaj;
      EXIT WHEN emp_cursor%NOTFOUND;
      INSERT INTO mesaje_pnu (rezultat)
      VALUES (v_mesaj);
    END LOOP;
    CLOSE emp_cursor;
  END LOOP;
  CLOSE dep_job;
  COMMIT;
END;
/
select * from mesaje_pnu;
```

FOR UPDATE, WHERE CURRENT OF

5. Să se dubleze valoarea salariilor celor angajaţi înainte de 1 ianuarie 1995, care nu câştigă comision.

```
DECLARE
  CURSOR before95 IS
    SELECT *
    FROM   emp_pnu
    WHERE comm IS NULL
```

```
    AND    hire_date <= TO_DATE('01-JAN-1995','DD-MON-YYYY')

-- ce efect ar avea urmatoarea comanda? Explicati.
--  DBMS_OUTPUT.PUT_LINE('Au fost actualizate '|| before95%ROWCOUNT || ' linii');
  COMMIT;  -- se permanentizeaza actiunea si se elibereaza blocarea
END;
/
```